30

© 한상훈 NestJS 백엔드 마스터

NestJS 백엔드 마스터

PART 2

Express와 Fastify 어댑터

제2부 NestJS 핵심 · 7장 수명주기와 플랫폼

Nest 아래의 HTTP 서버 바꾸기

© 한상훈 NestJS 백엔드 마스터

두 줄만 바꾼 뒤

터미널

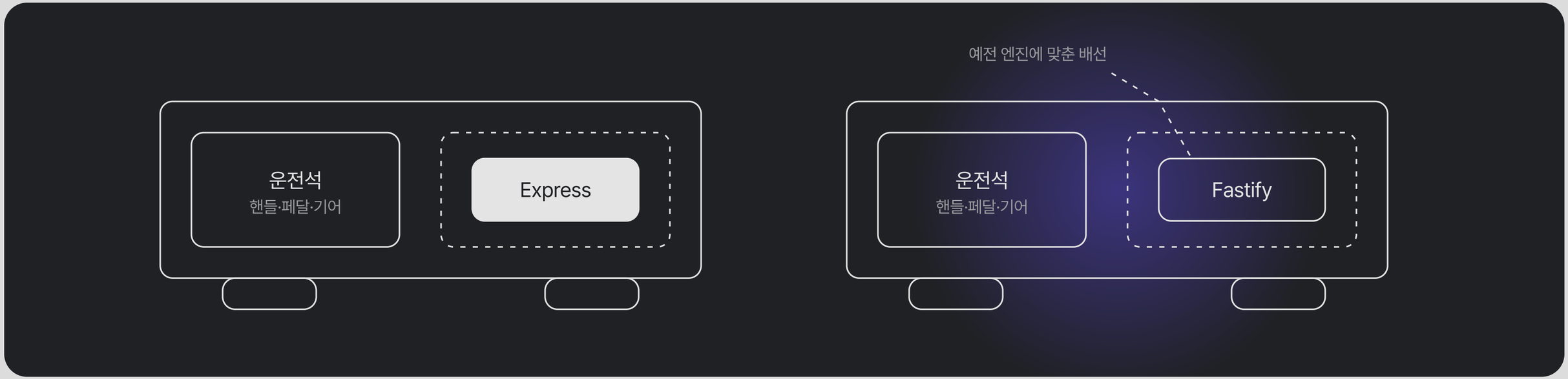
```
$ node dist/apps/ticketing/main.js
[Nest] 115612  - 2026. 10. 26. 오후 4:56:48     LOG [NestFactory] Starting Nest application...
...
[Nest] 115612  - 2026. 10. 26. 오후 4:56:48   ERROR [ExceptionHandler] TypeError: str is not iterable
    at parse (.../node_modules/path-to-regexp/dist/index.js:55:23)
    at process (.../node_modules/path-to-regexp/dist/index.js:263:56)
    at pathToRegexp (.../node_modules/path-to-regexp/dist/index.js:274:5)
    at Object.use (.../node_modules/@fastify/middie/lib/engine.js:34:26)
    at Object.use (.../node_modules/@fastify/middie/index.js:67:21)
    at FastifyAdapter.use (.../node_modules/@nestjs/platform-fastify/adapters/fastify-adapter.js:846:30)
    at NestApplication.use (.../node_modules/@nestjs/core/nest-application.js:254:26)
```

→ 바꾼 곳: `main.ts` 두 줄

→ 라우트 등록(Mapped) 전에 프로세스 종료

© 한상훈 NestJS 백엔드 마스터

같은 차체, 다른 엔진



→ 차체와 운전석: 컨트롤러, 서비스, 모듈, 데코레이터

→ 엔진: Express 또는 Fastify

→ 엔진룸에 직접 달아 둔 부품: 새 엔진에 안 맞음

© 한상훈 NestJS 백엔드 마스터

어댑터

→ 어댑터(adapter): Nest와 HTTP 서버 라이브러리 사이에 끼운 층

→ Nest는 어댑터에만 말함 (라우트 등록, 응답 보내기, 서버 열고 닫기)

→ 기본: `@nestjs/platform-express` (`nest new`가 설치)

→ 다른 선택: `@nestjs/platform-fastify`

© 한상훈 NestJS 백엔드 마스터

Fastify로 바꾸는 코드

터미널

```
npm install @nestjs/platform-fastify
```

main.ts

```
import { FastifyAdapter, type NestFastifyApplication } from '@nestjs/platform-fastify';

async function bootstrap() {
  const app = await NestFactory.create<NestFastifyApplication>(AppModule, new FastifyAdapter());
  app.use(requestId, requestLog);
  app.use(compression());
  // ... 전역 ValidationPipe, keepAliveTimeout, enableShutdownHooks
  await app.listen(process.env.PORT ?? 3000);
}
```

→ 바뀐 것: import 한 줄, `create`의 두 번째 인자와 타입 인자

© 한상훈 NestJS 백엔드 마스터

함수 두 개를 받은 app.use

```
app.use(requestId, requestLog);
```

→ Express의 `app.use`: 미들웨어 함수를 여러 개 받음

→ Fastify 어댑터의 `use` → `@fastify/middie`의 `use(path, fn)`: 첫 인자를 경로로 읽음

→ 경로 자리에 함수 → `path-to-regexp`에서 TypeError

```
app.use(requestId);
app.use(requestLog);
```

© 한상훈 NestJS 백엔드 마스터

부팅은 됐는데 전부 500

터미널

```
$ curl -i localhost:4030/concerts
HTTP/1.1 500 Internal Server Error
content-type: application/json; charset=utf-8

{"statusCode":500,"error":"Internal Server Error","message":"res.status(...).type(...).json is not a function"}
```

서버 로그

```
ERROR [ProblemDetails] TypeError: req.header is not a function
    at requestId (.../dist/apps/ticketing/main.js:517:20)
    at Holder.done (.../node_modules/@fastify/middie/lib/engine.js:149:11)
```

→ 공연 등록, 공연 목록, 예매, 없는 공연 조회: 모두 같은 500

→ 요청 ID 미들웨어: `req.header` (Express 요청 객체의 메서드)

→ 예외 필터: `res.status().type().json` (Express 응답 객체의 메서드)

→ 고객이 받은 본문: Fastify 기본 에러 응답, 내부 에러 문구가 그대로

© 한상훈 NestJS 백엔드 마스터

express에서 가져온 타입

터미널

```
$ grep -rn "from 'express'" apps/ticketing/src
common/domain-error.filter.ts:2:   import type { Request, Response } from 'express';
common/problem-details.filter.ts:3: import type { Request, Response } from 'express';
common/request-id.middleware.ts:2:  import type { NextFunction, Request, Response } from 'express';
common/request-log.middleware.ts:2: import type { NextFunction, Request, Response } from 'express';
common/staff-key.guard.ts:5:        import type { Request } from 'express';
common/timing.interceptor.ts:2:     import type { Request, Response } from 'express';
reservations/reservations.controller.ts:2: import type { Request } from 'express';
```

→ 미들웨어 둘, 예외 필터 둘, 가드, 인터셉터, 예매 컨트롤러: 일곱 파일

© 한상훈 NestJS 백엔드 마스터

tsc는 통과

터미널

```
$ npx tsc --noEmit -p apps/ticketing/tsconfig.app.json
tsc exit=0
```

→ Fastify로 바꾼 코드에서 실행

→ `import type`: 빌드 결과에서 사라짐

→ 타입에는 Express, 실제로 들어오는 객체는 Fastify 쪽 객체

© 한상훈 NestJS 백엔드 마스터

어댑터에 묶이는 코드

곳	Express에만 있는 것	Fastify에서
main.ts	`app.use(함수, 함수)`	부팅 중 TypeError
요청 ID 미들웨어	`req.header()`	모든 요청 500
예외 필터 둘	`res.status().type().json()`, `req.originalUrl`	필터가 깨짐
직원 키 가드	`req.header()`	매표소 판매 500
예매 컨트롤러	`@Req()`의 `req.get()`	예매 500
포스터 업로드	FileInterceptor (multer, Express 미들웨어)	415
처리 시간 인터셉터	(`req.headers`, `statusCode`만 읽음)	그대로 동작

© 한상훈 NestJS 백엔드 마스터

미들웨어는 Node 요청·응답만

request-id.middleware.ts

```ts
import type { IncomingMessage, ServerResponse } from 'node:http';

export function requestId(req: IncomingMessage, res: ServerResponse, next: () => void) {
  const id = (req.headers['x-request-id'] as string | undefined) ?? randomUUID();
  req.headers['x-request-id'] = id;
  res.setHeader('x-request-id', id);
  next();
}
```

→ Fastify의 미들웨어: Fastify 래퍼 대신 Node의 req, res를 받음 (공식 문서 Performance)

→ Express의 req, res도 Node 객체를 넓힌 것: 둘 다 쓰는 것만 씀

→ 요청 로그: `req.originalUrl ?? req.url`, `res.on('finish')`, `res.statusCode`

→ 직원 키 가드: `req.headers['x-staff-key']`

© 한상훈 NestJS 백엔드 마스터

예외 필터는 HttpAdapterHost로

problem-details.filter.ts

```ts
@Catch()
export class ProblemDetailsFilter implements ExceptionFilter {
  constructor(private readonly adapterHost: HttpAdapterHost) {}

  catch(exception: unknown, host: ArgumentsHost) {
    const { httpAdapter } = this.adapterHost;
    const req = host.switchToHttp().getRequest();
    const res = host.switchToHttp().getResponse();
    // ... status, body 만들기 (그대로)
    body.instance = httpAdapter.getRequestUrl(req);
    httpAdapter.setHeader(res, 'content-type', 'application/problem+json');
    httpAdapter.reply(res, body, status);
  }
}
```

→ 응답은 어댑터에게 맡김: `reply`, `setHeader`, `getRequestUrl`

→ DomainErrorFilter도 같은 방식

출처: NestJS 공식 문서 Exception filters (platform-agnostic 예제)

© 한상훈 NestJS 백엔드 마스터

예매만 남은 500

터미널

```
$ curl -X POST localhost:4030/reservations -A 'TicketApp/3.2 (iOS 18)' ...
{"type":"about:blank","title":"Internal Server Error","status":500,"detail":"요청을 처리하지 못했습니다.","instance":"/reservations"}
```

서버 로그

```
LOG [Timing] b591f975-fefe-4df9-8a45-9618d1a59203 ReservationsController.reserve 실패(TypeError) 1.0ms
ERROR [ProblemDetails] TypeError: req.get is not a function
```

reservations.controller.ts

```
reserve(@Body() body: CreateReservationDto, @Req() req: Request) {
  const userAgent = req.get('user-agent') ?? 'unknown';
```

→ 예매 기록에 User-Agent를 남김 (매크로 의심 예매를 나중에 찾으려고)

→ 필터가 고쳐져서: 내부 문구는 로그에만, 응답은 Problem Details

© 한상훈 NestJS 백엔드 마스터

Nest 데코레이터로 받기

reservations.controller.ts

```
@Post()
reserve(@Body() body: CreateReservationDto, @Headers('user-agent') userAgent = 'unknown') {
  return this.reservations.reserve(body.seatId, body.amount, body.buyer, body.channel, userAgent);
}
```

요청 쪽 기록과 서버 로그

```
[201]
[예매] A-3-15 결제 완료 (TicketApp/3.2 (iOS 18))
```

→ 받은 값: 예매 기록의 userAgent 칸, 결제 완료 로그의 괄호

© 한상훈 NestJS 백엔드 마스터

두 어댑터에서 같은 결과

요청	Express	Fastify
예매	201	201
없는 공연 99번	404, detail "공연 99번이 없습니다"	같음
매표소 판매 (key-kim)	201, soldBy "김매표"	같음
같은 좌석 다시 판매	409, SeatUnavailableError	같음
직원 키 없이 판매	401	같음
포스터 업로드	201	415

→ 고친 코드를 두 어댑터로 각각 띄워서 같은 요청

→ 포스터 업로드: Fastify용 multipart 처리가 따로 필요 (공식 문서 File upload)

© 한상훈 NestJS 백엔드 마스터

그대로인 라우트 표와 본문

부팅 로그

```
[RouterExplorer] Mapped {/reservations, POST} route
[RouterExplorer] Mapped {/reservations/refund, POST} route
[RouterExplorer] Mapped {/reservations/:id/receipt, GET} route
[NestApplication] Nest application successfully started
```

터미널

```
$ curl -X POST localhost:4030/concerts -H 'content-type: application/json' --data-binary @concert.json
{"id":1,"title":"가을 콘서트","venue":"올림픽홀"}
$ curl localhost:4030/concerts
[{"id":1,"title":"가을 콘서트","venue":"올림픽홀"}]
```

→ 부팅 로그의 Mapped 줄: Express 23개, Fastify 23개

→ 응답 본문: 두 어댑터에서 같은 출력

© 한상훈 NestJS 백엔드 마스터

다른 응답 헤더

Express	Fastify
`X-Powered-By: Express`	(없음)
`Content-Type: application/json; charset=utf-8`	`content-type: application/json; charset=utf-8`
`ETag: W/"3c-jbE9E2vrboUqmMR4G86jkdGKT3Q"`	(없음)
`Keep-Alive: timeout=65`	`Keep-Alive: timeout=65`

→ `curl -i localhost:4030/concerts`, 본문은 같음

→ X-Powered-By: 보안 점검에서 지우라는 단골 항목

→ timeout=65: `main.ts`에 적어 둔 keepAliveTimeout 65초. 그 줄을 지우면 Express 5, Fastify 72

"연결은 열어 두는 쪽이 싸고, 언제 닫을지는 양쪽이 맞춰야 합니다."

데이터 블록: express 5.3.0, fastify 5.12.7 (부록), 촬영 직전 확인

© 한상훈 NestJS 백엔드 마스터

듣는 주소

Express

```
  TCP    0.0.0.0:4030           0.0.0.0:0              LISTENING       180296
  TCP    [::]:4030              [::]:0                 LISTENING       180296
```

Fastify

```
  TCP    127.0.0.1:4030         0.0.0.0:0              LISTENING       216100
  TCP    [::1]:4030             [::]:0                 LISTENING       216100
```

→ `netstat -ano -p tcp`, 같은 `app.listen(4030)`

→ `0.0.0.0`, `[::]`: 이 컴퓨터의 모든 주소

→ `127.0.0.1`, `[::1]`: 이 컴퓨터 안에서만

© 한상훈 NestJS 백엔드 마스터

랜 주소로 부르면

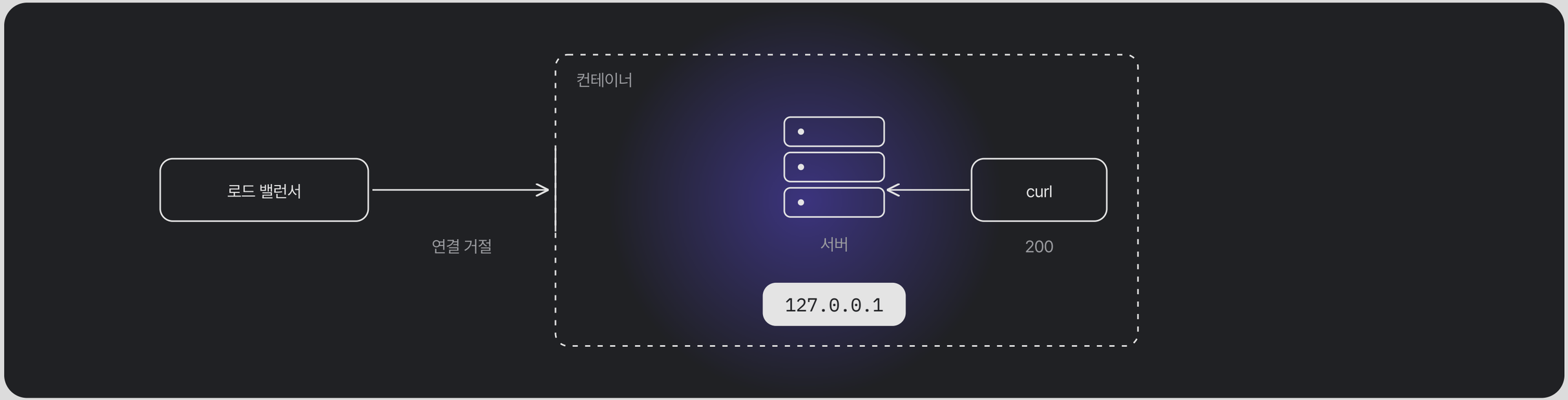
```
Express
$ curl http://192.168.45.220:4030/concerts
[{"id":1,"title":"가을 콘서트","venue":"올림픽홀"}]
[200]

Fastify
$ curl http://192.168.45.220:4030/concerts
curl: (7) Failed to connect to 192.168.45.220 port 4030 after 2040 ms: Could not connect to server
```

→ 같은 PC, 랜 카드 주소로 호출

© 한상훈 NestJS 백엔드 마스터

컨테이너 안의 127.0.0.1



→ 컨테이너 안의 127.0.0.1: 그 컨테이너 자신

→ 포트를 열어도 바깥 요청은 컨테이너의 다른 주소로 도착

→ 컨테이너 안 curl은 200, 로드 밸런서는 연결 거절 (가정)

© 한상훈 NestJS 백엔드 마스터

모든 주소에서 듣기

main.ts

```
await app.listen(process.env.PORT ?? 3000, '0.0.0.0');
```

터미널

```
  TCP    0.0.0.0:4030           0.0.0.0:0              LISTENING       104580
$ curl http://192.168.45.220:4030/concerts
[{"id":1,"title":"가을 콘서트","venue":"올림픽홀"}]
[200]
```

출처: NestJS 공식 문서 Performance (Fastify)

© 한상훈 NestJS 백엔드 마스터

200KB 본문

터미널

```
$ curl -X POST localhost:4030/concerts -H 'content-type: application/json' --data-binary @big200k.json
Express  [413] {"type":"about:blank","title":"Payload Too Large","status":413,
                "detail":"request entity too large","instance":"/concerts"}
Fastify  [400] {"type":"about:blank","title":"Bad Request","status":400,
                "errors":["title must be shorter than or equal to 100 characters"],"instance":"/concerts"}
```

→ 제목 200KB짜리 공연 등록 본문

→ Express: body parser 상한(100KB)에서 거절

→ Fastify: 상한 1MiB 안이라 본문을 다 읽고 JSON으로 바꾼 뒤, 검증 파이프(MaxLength 100)에서 거절

© 한상훈 NestJS 백엔드 마스터

2MB 본문과 깨진 JSON

```
2MB 본문
Express  [413] "detail":"request entity too large"
Fastify  [413] "detail":"Request body is too large"

깨진 JSON: -d '{"title":'
Express  [400] "detail":"Unexpected end of JSON input"
Fastify  [400] "detail":"Body is not valid JSON but content-type is set to 'application/json'"
```

→ 본문 모양: 둘 다 우리 필터가 만든 Problem Details (type, title, status, detail, instance)

→ 다른 것: detail 문구

© 한상훈 NestJS 백엔드 마스터

상한을 적어 두기

main.ts

```ts
const app = await NestFactory.create<NestFastifyApplication>(
  AppModule,
  new FastifyAdapter({ bodyLimit: 100 * 1024 }),
);
```

터미널

```
$ curl ... --data-binary @big200k.json
[413] {"type":"about:blank","title":"Payload Too Large","status":413,"detail":"Request body is too large","instance":"/concerts"}
```

→ 기본 상한: Express(body-parser) 100KB, Fastify 1MiB

→ 바꾸기 전과 같은 상한을 코드에 적음

© 한상훈 NestJS 백엔드 마스터

처리량 재기

터미널

```
npx autocannon -c 100 -d 10 localhost:4030/concerts
npx autocannon -c 100 -d 10 -m POST -H content-type=application/json \
  -i rsv-bench.json localhost:4030/reservations
```

→ 연결 100개, 10초, 경로마다 세 번 (앞에 5초씩 예열)

→ 두 어댑터 모두 고친 코드: 요청 ID·로그 미들웨어, 가드, 인터셉터, 검증 파이프, 요청마다 로그 두 줄

→ 부하 도구와 서버가 같은 PC: 절댓값은 운영과 다름, 두 어댑터 비교용

→ 예매: 메모리 가짜 결제사, 결제마다 20ms 대기 (`FAKE_PG_DELAY_MS=20`)

데이터 블록: Windows, Node v24.11.0, autocannon 8.0.0, 같은 PC (부록), 촬영 직전 확인

© 한상훈 NestJS 백엔드 마스터

공연 목록

어댑터	초당 요청 (평균)	평균 지연
Express 1회	12,193.6	7.71ms
Express 2회	12,916	7.24ms
Express 3회	12,969.1	7.21ms
Fastify 1회	15,081.1	6.13ms
Fastify 2회	14,181.6	6.54ms
Fastify 3회	13,911.6	6.69ms

→ 메모리 배열을 바로 돌려주는 경로: 약 1.1~1.2배

© 한상훈 NestJS 백엔드 마스터

결제사를 20ms 기다리는 예매

어댑터	초당 요청 (평균)	평균 지연
Express 1회	3,095.8	31.94ms
Express 2회	2,926.4	33.76ms
Express 3회	3,067.2	32.18ms
Fastify 1회	2,659.9	37.21ms
Fastify 2회	3,102	31.83ms
Fastify 3회	3,167.6	31.18ms

→ 차이 없음 (회차끼리의 차이가 더 큼)

© 한상훈 NestJS 백엔드 마스터

가장 느린 곳



→ 공연 목록: 미들웨어, 가드, 인터셉터, 로그 두 줄은 어댑터와 상관없이 같은 일

→ 예매: 결제사 20ms가 응답 시간의 대부분

"요청 하나는 여러 곳을 거치고, 그중 가장 느린 곳이 응답 시간을 정합니다."

© 한상훈 NestJS 백엔드 마스터

그대로인 것과 바뀌는 것

그대로	바뀜
컨트롤러·서비스·모듈	Express 메서드를 부른 미들웨어·필터·가드
`@Body·@Param·@Query·@Headers`	`@Req·@Res`로 받은 객체, Express용 패키지(multer 등)
라우트 표, 응답 본문	듣는 주소, 본문 상한, 에러 문구, 응답 헤더
직접 적어 둔 값(keepAliveTimeout 65초)	안 적어 둔 기본값
수명주기 훅, enableShutdownHooks	처리량 (경로에 따라)

© 한상훈 NestJS 백엔드 마스터

이 프로젝트의 main.ts

main.ts

```ts
async function bootstrap() {
  const app = await NestFactory.create(AppModule);
  app.use(requestId, requestLog);
  app.use(compression());
  app.useGlobalPipes(new ValidationPipe({ whitelist: true, forbidNonWhitelisted: true, transform: true }));
  const server = app.getHttpServer();
  server.keepAliveTimeout = 65_000;
  app.enableShutdownHooks();
  await app.listen(process.env.PORT ?? 3000);
}
```

→ Express 유지, `@nestjs/platform-fastify` 제거

→ 남긴 것: 예매 컨트롤러의 `@Headers('user-agent')`

→ 미들웨어·필터·가드를 고친 코드: 가지 `demo/s030-fastify`에 보관

→ 다시 볼 때: 메모리에서 바로 답하는 경로가 병목이 될 때

© 한상훈 NestJS 백엔드 마스터

어댑터를 바꿀 때

"Nest 데코레이터로 값을 받는 코드는 어댑터를 바꿔도 그대로 돌고, 바뀌는 건 요청 객체와 기본값입니다."